

SDE 数据合成流水线

为更好地捕捉现实时空动态，我们的数据合成流水线包含三个关键设计选择：

- 需求源与传播节点**：我们将节点分为需求源节点和传播节点。具体地，我们强制需求源节点的漂移函数 f_i 为正弦或均值回复型，而传播节点仅分配均值回复型漂移函数。需求源节点的漂移函数可以是时变的。我们也尝试了更多样化的漂移函数分配，但效果不佳。通过这种设计，需求源节点的趋势通过图传播到其他传播节点，而传播节点本身仅具有均值回复趋势，因此主要受邻居影响。这种设置模拟了现实场景。例如，在包含三个节点（分别代表住宅区、高速公路和商业区）的交通设置中，住宅区和商业区是需求源节点，因为交通自然源于此（例如，人们开车离开车库），而高速公路是传播节点，其交通主要来自其他区域，并非本地产生。
- 时变邻接矩阵**：为了模拟不同时间的不同空间依赖，我们将图邻接矩阵设为时变。使用同一示例，早晨交通应主要从住宅区流向道路再到商业区，而傍晚方向相反。因此，邻接矩阵中的边权应随时间变化，例如在早晨增加住宅区到道路的边权，在夜间减少，对道路到住宅区的边则相反。
- 传播时间滞后**：此外，我们引入时间滞后以模拟传播延迟，因为节点之间的交互并非瞬时。每条边被分配一个时间滞后以反映传播速度。

通过这三个组件，我们期望我们的流水线能够模拟与真实世界系统观察到的动态相近的时空数据。

SDE 工作流程：

第一阶段：剧本创作与结构化（把“人话”变成“表格”）

目标：将模糊的文字描述，转化为机器可读的“零件清单（JSON）”。

- Agent 1（场景生成）**：这是剧作家。它根据要求（比如“写一个3节点的交通早高峰场景”），生成一段详细的自然语言描述。它必须明确写出：节点谁是需求源、谁是传播节点、边的方向、时滞（延迟多久）、波峰发生在第几步。
 - 关键约束**：它必须确保“需求源有出边”、“传播节点振幅为0”等硬规则。
- Agent 2（场景解析）**：这是翻译官。它把Agent 1的长篇大论，精准翻译成一份**结构化的JSON文件**。这份JSON里清晰列出了：节点ID、类型、边（含 `time_lag`）、每个节点在不同时间段的波形类型（正弦波还是均值回归）、基线值、峰值时间等。
- Judge 1（场景逻辑校验）**：这是**第一道质检员**。它拿着原始剧本（Agent 1）和翻译稿（Agent 2），严格核对两件事：
 - 翻译是否漏词**：剧本里写的边，JSON里都有吗？数值提取错了吗？
 - 剧本逻辑是否自治**：比如剧本说“Node 0→Node 1延迟1步，Node 1→Node 2延迟1步”，那么早高峰到达Node 2的时间，必须比到达Node 1晚1步。如果写成了同时到达，Judge 1会立刻打回，并指明是Agent 1的逻辑错了。
 - 只有Judge 1判定“批准（Approve）”，JSON才会流入下一关；否则打回重写，形成闭环。**

第二阶段：数学参数实例化（把“表格”变成“公式系数”）

目标：把JSON里的物理描述，填入SDE公式 $dX_i(t) = \dots$ 的具体槽位中。

- **Agent 3 (SDE参数生成)**：这是数学老师。它将JSON里的“正弦波”映射为具体的 f_i 函数，并设定漂移系数 (κ, σ) 。它严格遵循节点类型规则：
 - 若节点是**需求源**：赋予 `sinusoidal`（带振幅 A 、频率 ω 、相位 φ ）或 `constant` 漂移项。
 - 若节点是**传播节点**：强制设为 `mean_reverting`（均值回归，振幅强制为0）。
 - 同时，给每个节点分配耦合强度 λ_i （传播节点给高值，以便传递信号）。
- **Agent 4 (时变邻接生成)**：这是交通调度员。它专门负责设定公式第三项中的 **邻接权重** $A_{ji}(t)$ 。它将JSON里的“早高峰（7-9点）效应强”，映射为在时间步 14-19 内，特定边的**瞬时倍数 (Multiplier)**。例如，平时这条边权重是1.0，早高峰瞬间暴涨到15（**strong** 效应）。
- **Judge 2 (参数与仿真校验)**：这是**第二道、也是最硬核的质检员**。它不看文本，而是要求系统**先用当前参数跑一遍仿真 (Simulate)**，生成初步的波形图，然后Judge 2盯着这张图看：
 1. **信号传对了吗？** 如果需求源Node 2在Step 14出现了波峰，但下游传播节点在Step 15毫无反应，Judge 2会判定“耦合强度不够”，**打回建议**：“请把Node 2→Node 1的Multiplier从10提高到15。”
 2. **噪声合理吗？** 如果曲线过于平滑像假数据，Judge 2会要求增加随机噪声 (σ) ；如果噪声把波峰淹没了，则要求降低 σ 。
 3. **传播节点违规了吗？** 检查传播节点是否有独立于邻居的自生波峰，若有，强制打回修正Agent 3的参数。
 - 只有Judge 2批准，这组参数才算最终定稿。

第三阶段：最终数值模拟与人工精选（生成“成品”）

目标：用最终定稿的参数，生成高质量的成品数据。

- **仿真器执行**：拿着Judge 2批准后的最终参数 $(f_i, \lambda_i, A_{ji}(t), \tau_{ji})$ ，代入SDE公式，进行**欧拉-丸山 (Euler-Maruyama) 数值积分**。这一步生成的就是我们需要的**多节点离散时序数字 $(X_i(t))$** 。
- **人工把关 (Human Validation)**：论文生成了1200套这样的“文本描述+时序数据”配对样本。但他们不放心，**雇人（或作者亲自）进行了人工肉眼检查 (图7)**，检查数据是否真的和描述一致。最终淘汰了136套“低质量”的，**仅保留1064套“高质量”样本**，作为ST-Bench和ST-Align的原始题库。

ST-Align 旨在通过基础理解问题在训练期间将文本表示与时空数据对齐：**时间问题**关注节点级时间动态，**空间问题**探测图连通性和影响路径，**时空问题**联合推理传播延迟和时变空间交互。具体地，我们基于 SDE 参数智能体、时变邻接智能体的输出以及模拟的时空数据，使用预定义模板（见附录 J）生成 QA 对。

STReasoner 模型整体架构：

训练设置及数据集：

前两个阶段的训练只使用有图结构的数据，第三阶段即使用有图结构的数据也使用无图的数据。

实验结果：

现有 LLM 的性能。专有模型在所有任务上均表现出较强的零样本性能，尤其在预测任务上。然而，它们的大模型规模导致计算成本显著提高。相比之下，大多数轻量级开源模型和时间序列 LMs/RMs 的性能明显低于专有模型。其中，时间序列 RM 持续优于时间序列 LM，表明这些任务需要显式推理能力。从成本-性能角度看，将时间序列提示为图像通常比纯文本更有效，因为它在更低的标记成本下获得了更好的性能。

卓越性能。STReasoner 在所有四个任务上均取得了开源模型和时间序列模型中的最佳整体性能。与专有模型相比，STReasoner 以平均仅为专有模型 0.004 倍的成本实现了 T1（病因）、T2（实体）和 T3（相关性）任务上分别 17%、135% 和 40% 的平均提升，同时在 T4（预测）任务上仅相差 2.6%，达到可比性能。这些结果验证了时间序列编码器和所提出的三阶段训练策略在时空推理中的有效性。

时间序列模态的研究。为了研究时间序列模态设计和使用独立时间序列编码器的效果，我们将相同的训练策略（包括 SFT 和 S-GRPO）应用于将时间序列提示为文本或图像的模型。基于图像的提示在依赖全局形状的任务（如病因、实体和相关性推理）上表现良好，但在需要精确数值信息的预测任务上不如基于文本的提示。总体而言，仅使用文本模态或基于图像的多模态的模型表现显著逊于 STReasoner。这一结果表明，通过专用编码器显式编码时间序列对于保留全局形状和数值信息以进行时空推理是必要的。

RL 训练曲线。图 4 展示了 STReasoner 的 RL 训练动态。如图 4a 所示，准确率奖励总体呈上升趋势，表明模型在 RL 训练期间逐步提高生成正确答案的能力。类似地，图 4b 显示空间奖励稳步增加，表明模型在 S-GRPO 下越来越依赖空间基础的推理行为。值得注意的是，图 4c 中的响应长度在 RL 训练早期下降，随后逐渐增加并稳定。这种模式可能反映了学习到的推理策略的转变：模型最初偏离次优的 SFT 推理模式，最终收敛到更稳定和有效的推理策略。

S-GRPO 对空间推理使用的影响。为了进一步评估 S-GRPO 是否鼓励模型在推理中主动依赖空间结构，我们使用 GPT-5.2 测量了在需要空间推理的问题上显式使用空间信息的响应百分比。评估提示见附录 L。如图 5 所示，使用 S-GRPO 训练的 STReasoner 在所有任务上均表现出比普通 GRPO 高得多的空间推理使用率。这些结果表明 S-GRPO 不仅提高了最终准确率，更重要的是将模型的推理行为转向空间基础的策略。

RL 期间空间信息使用的影响。为了进一步检查 S-GRPO 是否提高了模型利用空间知识的能力，我们在不同 RL 训练步骤下评估模型检查点，在两种输入设置下：有图结构 and 无图结构。具体地，我们报告步骤 1-51 在任务 1 上的准确率，并测量两种设置之间的性能差距 Δ 。如表 5 所示，整体准确率在 RL 训练期间稳步提高。更重要的是，空间输入带来的性能增益变得越来越显著。差距 Δ 从步骤 1 的 0.5 增长到步骤 51 的 6.2，表明模型逐步更多地依赖空间结构。这一结果直接证明 S-GRPO 增强了模型利用空间信息的能力，而不仅仅是提高整体准确率。

敏感性分析。图 6 展示了空间奖励幅度 α 的敏感性分析。中间值 ($\alpha = 0.2$ 和 0.1) 产生持续强劲的性能，而极端设置 ($\alpha = 1, 0.5, 0$) 则降低一个或多个指标。较大的 α 使空间奖励主导优化，使模型偏离答案正确性，而移除 α 则无法鼓励空间感知的推理。

5.2 真实数据上的零样本结果

为了评估 STReasoner 在真实世界时空数据上的实际零样本性能，我们基于 CausalRivers (Stein 等, 2025) 构建了一个因果 QA 数据集。具体地，给定底层因果图，如果存在从节点 A 到节点 B 的有向边，则定义从 A 到 B 的因果效应，并为每条这样的边生成一个 QA 实例，询问节点 A 是否对节点 B 有因果效应。我们在严格的零样本设置下评估 STReasoner，即未在该数据集上进行任何微调。如表 2 所示，尽管仅在合成数据上训练，STReasoner 在该真实世界基准上显著优于两个大型专有模型。这些结果表明了强大的泛化能力，并验证了我们的合成数据构建和训练策略的有效性。

5.3 消融研究

Align 是否有帮助？Align 旨在将文本指令与时间序列表示对齐，为后续推理训练提供基础。如表 3 所示，仅使用 Align 训练的模型在所有任务上表现较差。这是因为 Align 目标并非直接针对下游推理任务优化，因此其本身不能产生推理能力。比较 SFT+S-GRPO 与 STReasoner，添加 Align 使最终性能平均提高了 6.3%。这表明当与 SFT 和 S-GRPO 结合时，对齐提供了互补的益处。

SFT 是否有帮助？ SFT 在建立指令遵循和基础推理能力方面起着关键作用。跳过 SFT 而直接应用 S-GRPO 的模型表现明显更差，这是由于在弱推理初始化下，滚动过程中的奖励信号稀疏且不稳定。

S-GRPO 是否有帮助？ 在 SFT 之后，RL 变得有效。比较 Align+SFT+GRPO 与 Align+SFT，显示 RL 进一步提高了准确率。此外，将 STReasoner 与 Align+SFT+GRPO 比较，显示用所提出的 S-GRPO 替换 GRPO 平均提升了 5.10%，证明了空间感知 RL 在时空推理中的有效性。